

SYSTEM DESIGN

Page 1

LLD & HLD of Amazon / Flipkart E-Commerce

① WHAT IS E-COMMERCE SYSTEM?

- An E-commerce system allows users to browse products, add to cart, place order and make payment online.
- Examples : Amazon, Flipkart, Myntra, etc.



② CORE FUNCTIONAL REQUIREMENTS

- User Management (Sign up, Login, Profile)
- Product Management (Add, Update, Delete)
- Search & Filter Products
- Add to Cart & Wishlist
- Place Order & Checkout
- Payment Processing
- Order Tracking
- Notifications (Email, SMS, Push)
- Reviews & Ratings

Customer satisfaction is the key!

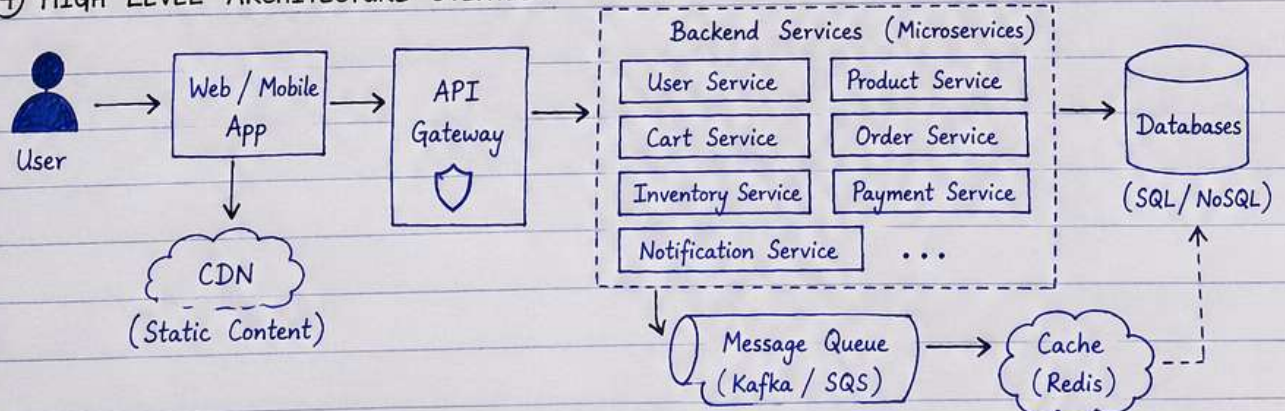
③ NON-FUNCTIONAL REQUIREMENTS

- Scalability - Handle millions of users & requests
- Availability - 99.99%, uptime
- Performance - Low latency, fast response
- Reliability - Fault tolerant, data consistency
- Security - Data protection, secure transactions
- Maintainability - Easy to update & deploy
- Usability - Simple & intuitive UI

Target Metrics

- 1M+ Users / day
- 10K+ Orders / sec
- < 200 ms Response Time

④ HIGH LEVEL ARCHITECTURE OVERVIEW



* This is a high level view. We will deep dive into each component in next pages.

② HIGH LEVEL DESIGN (HLD)

- HLD focuses on the **overall architecture** of the system.
- We break the system into independent services (**microservices**).
- Each service is responsible for a **specific business capability**.
- Services communicate with each other through **APIs / Events**.

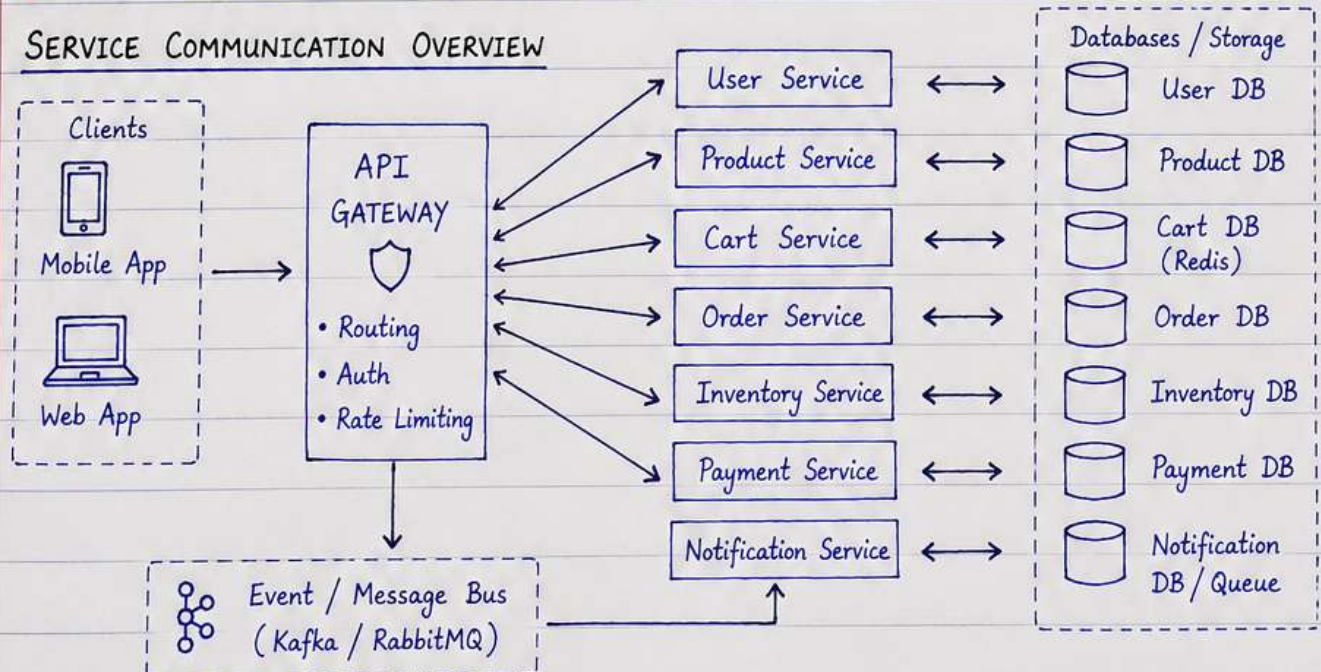
MAJOR MICROSERVICES

- ① **User Service** - Handle user registration, login, profile, addresses, authentication.
- ② **Product Service** - Manage product catalog, categories, product details, price, images.
- ③ **Cart Service** - Manage user cart, add/remove items, cart persistence.
- ④ **Order Service** - Create orders, manage order lifecycle, order history.
- ⑤ **Inventory Service** - Manage stock, reserved stock, stock update across warehouses.
- ⑥ **Payment Service** - Handle payments, refunds, payment status, gateways.
- ⑦ **Notification Service** - Send emails, SMS, push notifications.

Each service owns its data. ★

★
Decoupled services make the system scalable & maintainable.

SERVICE COMMUNICATION OVERVIEW



- * This HLD ensures **loose coupling, scalability, high availability and easy independent deployment of services.**

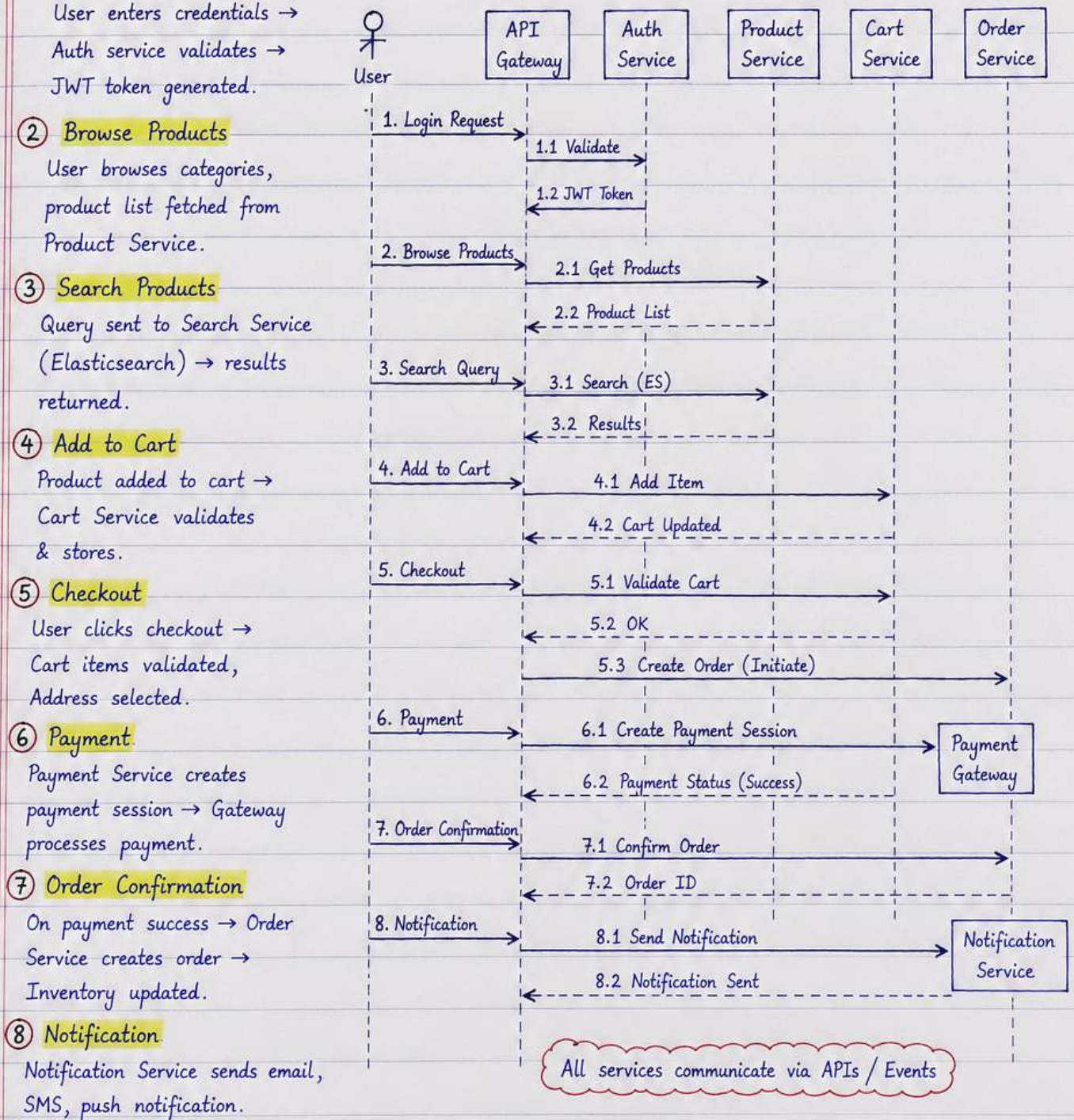
③ COMPLETE REQUEST FLOW (End to End)

- This flow shows how a user goes from **browsing to order placement**.

① User Login

User enters credentials →
Auth service validates →
JWT token generated.

SEQUENCE DIAGRAM (High Level)



KEY POINTS

- All requests go through **API Gateway** (single entry point).
- Services are loosely coupled and communicate via **APIs / Events**.
- Data is validated at every step (Auth, Cart, Inventory, Payment).
- Order is created only after **successful payment**.
- Notifications are **asynchronous** (via Queue).

TECH USED

- JWT for Auth
- REST APIs
- Elasticsearch for Search
- Kafka / SQS for Events
- Redis for Cache / Cart
- MySQL / DynamoDB for DB

* **Tip:** Always think in user flow first, then break into services.

④ DATABASE DESIGN

- We use a combination of **SQL** (Relational DB) and **NoSQL** (Non-Relational DB) based on the use case.

SQL vs NoSQL

SQL (Relational DB)	NoSQL (Non-Relational DB)
- Structured data - ACID Transactions - Joins, complex queries - Examples: MySQL, PostgreSQL	- Unstructured / Semi-structured data - High scalability - High performance - Examples: MongoDB, Cassandra, Redis

We use both SQL and NoSQL to handle scale and performance!



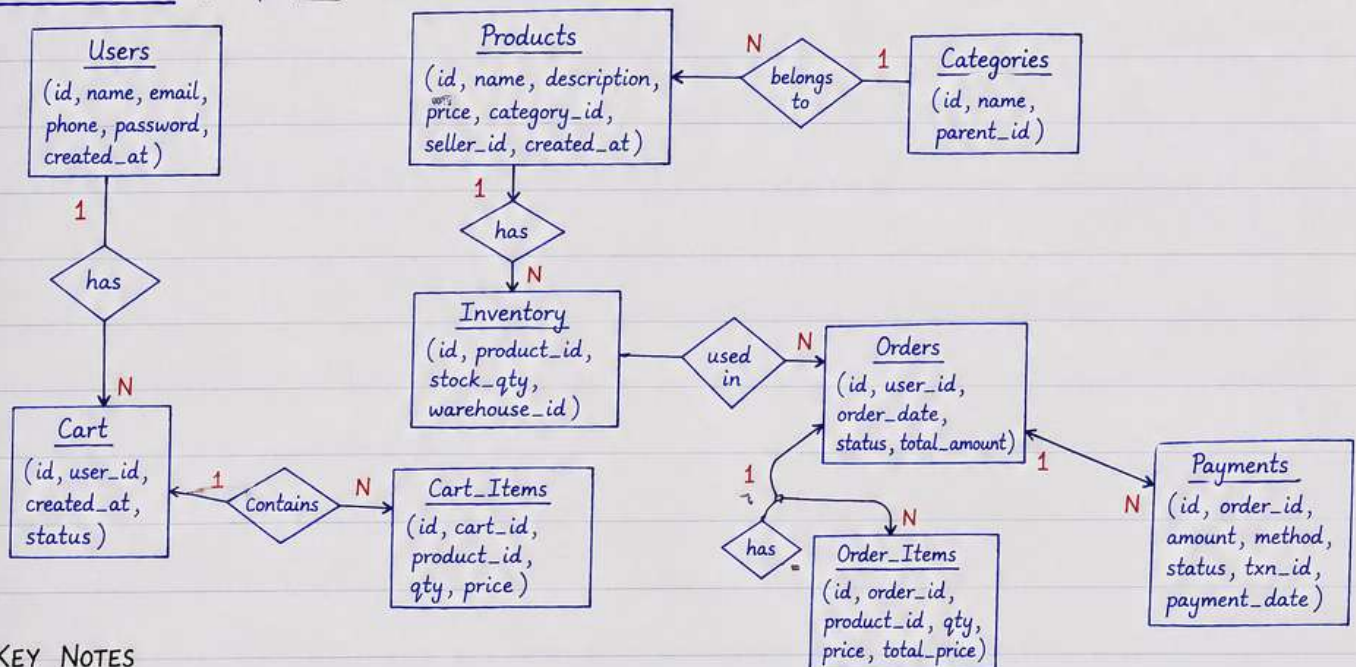
MAIN TABLES (SQL)

- Users** - Stores user information.
- Products** - Stores product catalog details.
- Inventory** - Stores stock information for products.
- Orders** - Stores order header information.
- Order_Items** - Stores items in each order.
- Cart** - Stores items added in user cart.
- Payments** - Stores payment & transaction details.

Other Important Tables (Optional)

- Addresses
- Categories
- Reviews
- Coupons
- Shipments
- Returns
- Refunds

ER DIAGRAM (Simplified)



KEY NOTES

- Users** → can have multiple orders.
- Each **Order** → can have multiple items.
- Inventory** → tracks stock for each product.
- Payments** → can have multiple attempts for an order.
- Use **Indexing** on (product_id, user_id, order_id) for fast queries.

Where to use NoSQL?

- Product Reviews
- User Activity Logs
- Session Management
- Caching (Redis)
- Analytics Events

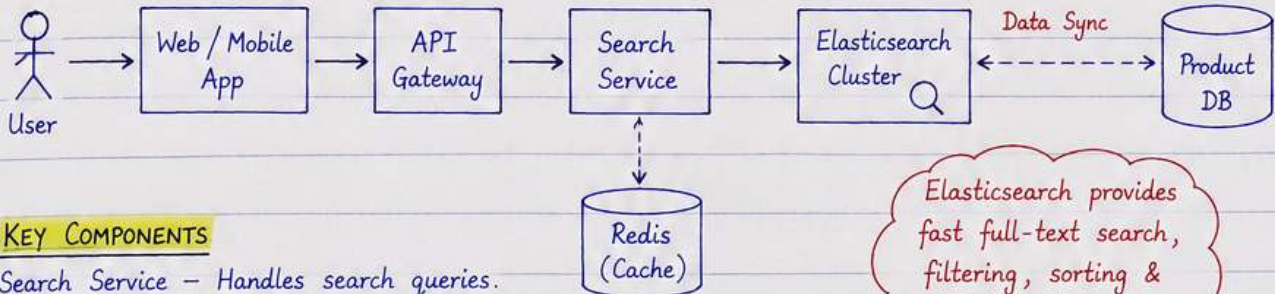
* Design may vary based on business requirements.

5] PRODUCT SEARCH SYSTEM

Search is a critical feature in any e-commerce platform.

We need fast, relevant and scalable search.

1] SEARCH ARCHITECTURE. (High Level)



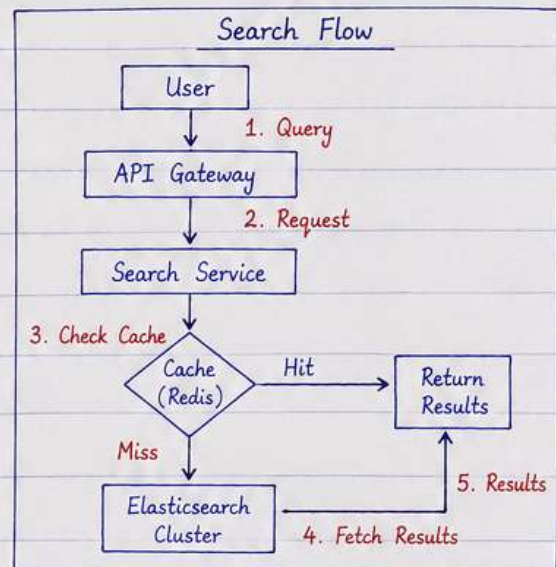
Elasticsearch provides fast full-text search, filtering, sorting & aggregation. 😊

2] KEY COMPONENTS

- Search Service - Handles search queries.
- Elasticsearch - Stores indexed product data.
- Product DB - Source of truth.
- Redis - Caches frequent queries / suggestions.
- Indexer / Data Sync - Syncs DB → ES.

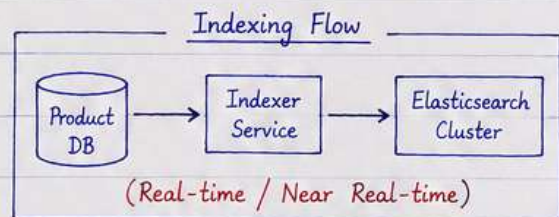
3] HOW IT WORKS (Flow)

1. User enters search query (e.g. "iphone 15").
2. API Gateway forwards request to Search Service.
3. Search Service checks Redis cache.
 - If cache hit → return results.
 - If cache miss → query Elasticsearch.
4. Elasticsearch returns relevant results.
5. Results are sent back to user.
6. Frequently searched queries are cached in Redis.



4] INDEXING PROCESS

- Products are fetched from Product DB.
- Indexer transforms data into JSON documents.
- Documents are indexed in Elasticsearch.
- Indexing can be real-time or near real-time.



5] SEARCH FEATURES

- Full Text Search - Search by name, description, brand, etc.
- Filters - By category, brand, price range, ratings, etc.
- Sorting - By relevance, price (low-high), price (high-low), ratings.
- Autocomplete / Suggestions - Show suggestions as user types.
- Faceted Search - Show filter counts (e.g. brand A (120), brand B (80)).
- Pagination - Handle large result sets.

6] TECH USED

- Elasticsearch
- Kibana (Monitoring)
- Logstash / Beats (Optional)
- Redis
- Kafka (For async indexing)

Example Query

```

GET /products/_search
{
  "query": {
    "multi_match": {
      "query": "iphone 15",
      "fields": ["name", "brand", "description"],
      "fuzziness": "AUTO"
    }
  },
  "sort": [{"_score": "desc"}],
  "size": 20,
  "from": 0
}
  
```

* Good search improves user experience and increases sales conversions! 😊

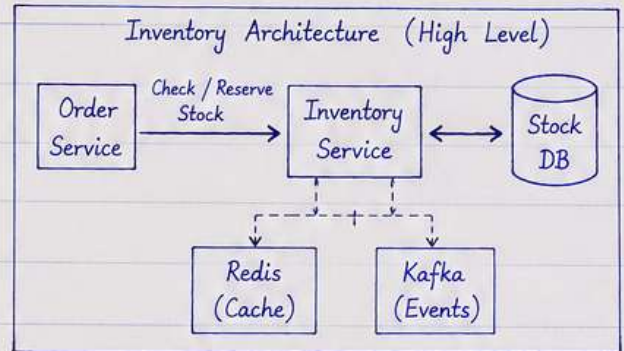
6 INVENTORY MANAGEMENT

- Inventory Management ensures that products are available in the **right quantity** at the **right time**.
- It prevents **over-selling** and ensures smooth order fulfillment.

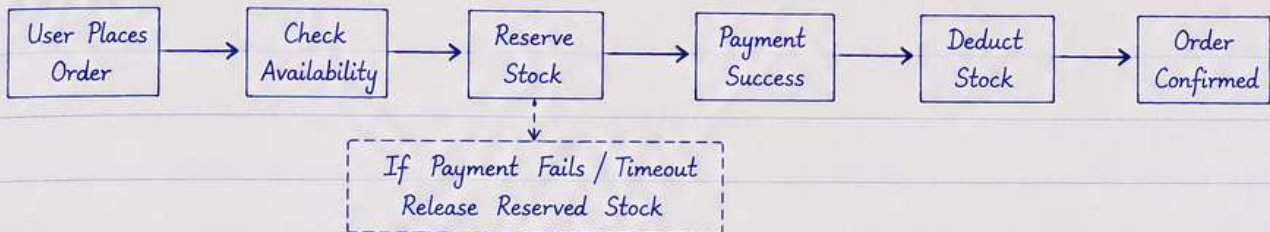


1 COMPONENTS

- Inventory Service - Manages stock, reservation, deduction.
- Warehouse Service - Manages multiple warehouses & locations.
- Stock DB - Stores real-time stock information.
- Cache (Redis) - For fast reads and stock locks.
- Message Queue (Kafka) - For async inventory updates.

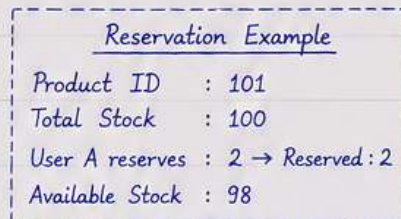


2 STOCK FLOW



3 STOCK RESERVATION

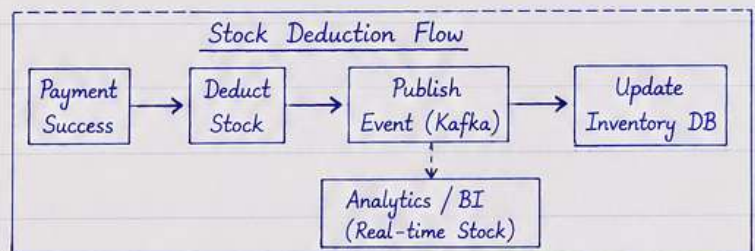
- When user places order, stock is **reserved** for a limited time (e.g., 15 minutes).
- Prevents other users from purchasing the same items.
- If payment is not completed in time, reservation is released.



Reservation is like "holding the stock" 😊

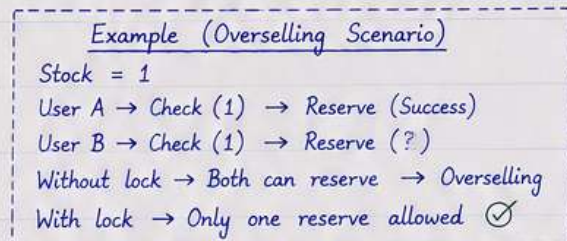
4 STOCK DEDUCTION

- After successful payment, reserved stock is **deducted** from available stock.
- An inventory event is published to Kafka.
- Inventory DB is updated asynchronously.



5 RACE CONDITION & SOLUTION

- Problem: Multiple users try to buy last item at same time.
- Solution:
 - ✓ Use DB transactions with **row-level locking**.
 - ✓ Or use **Redis distributed lock**.
 - ✓ Or use **Optimistic locking** (version field).



6 WAREHOUSE MANAGEMENT

- Multiple warehouses in different locations.
- Stock is allocated based on nearest warehouse.
- Helps in faster delivery and lower shipping cost.

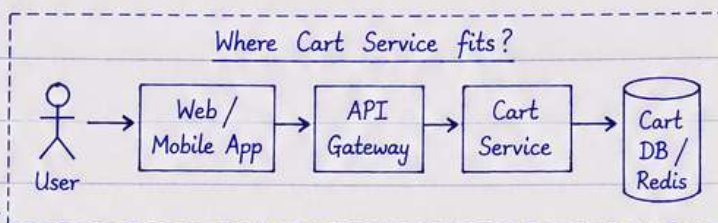


* Accurate inventory management = Happy Customers + No Revenue Loss 😊

7 CART SERVICE (LLD)

Cart Service is responsible for managing items that users want to buy later.

It provides a temporary storage before checkout.

**1 CART APIs**

- POST /cart/add → Add item to cart
- GET /cart → Get cart items
- PUT /cart/update → Update item quantity
- DELETE /cart/item/{itemId} → Remove item
- DELETE /cart/clear → Clear the cart
- POST /cart/merge → Merge guest cart with user cart
- GET /cart/count → Get cart item count

Guest users have a cart using a token / cookie.
After login, cart is merged with user cart.

**2 CART DATABASE DESIGN**

Cart Table (SQL / NoSQL)

Field	Type	Description
id	BIGINT (PK)	Primary Key
user_id	BIGINT (Nullable)	User ID (Null for guest)
guest_token	VARCHAR	Token for guest users
product_id	BIGINT	Product ID
quantity	INT	Quantity of the product
price	DECIMAL	Price at the time of adding
added_at	TIMESTAMP	When item was added
updated_at	TIMESTAMP	When item was last updated

Redis Cache Structure (Example)

Key: cart:{userId or guestToken}

Value: Hash

Field	Example
productId: 101	{qty: 2, price: 499}
productId: 102	{qty: 1, price: 999}
productId: 103	{qty: 3, price: 299}

Redis is used for fast read/write and session cart storage.

3 GUEST CART VS LOGGED-IN CARTGUEST CART

- Identified by guest_token (stored in cookie)
- Stored in Redis / DB
- Lost if cookie is cleared

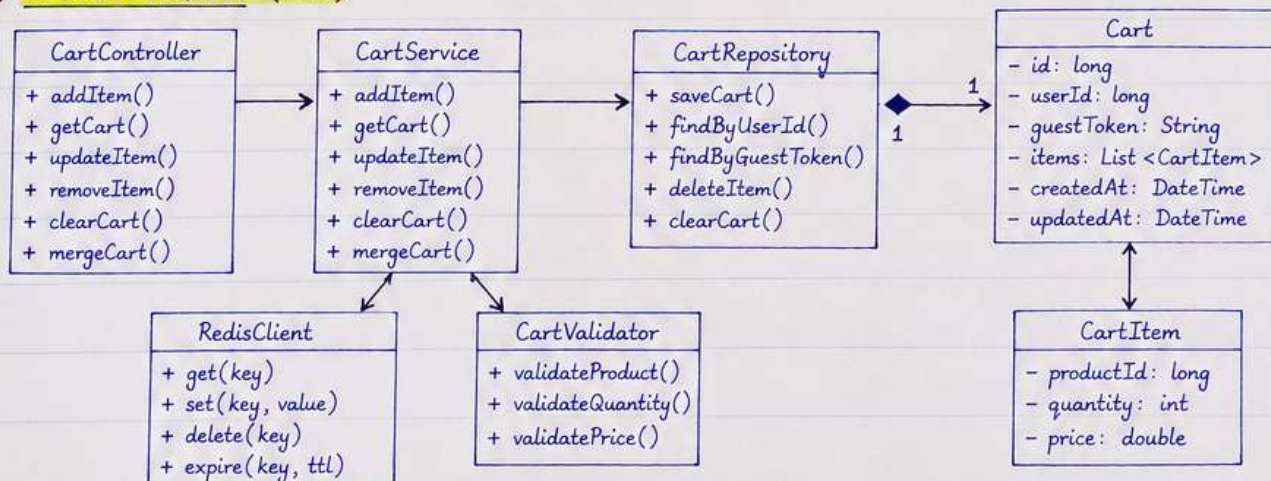
VS

LOGGED-IN CART

- Identified by user_id
- Persistent across devices
- Stored in DB + Redis (cache)
- Survives even if cookie is cleared

MERGE STRATEGY

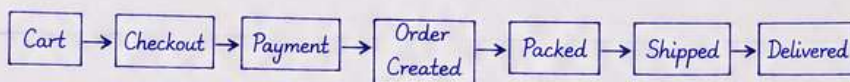
- On login, fetch guest cart
- Fetch user cart
- Merge both (sum of quantities for same product)
- Save final cart to DB/Redis
- Invalidate guest token

4 CLASS DIAGRAM (LLD)

- * Cart Service should be fast, scalable and consistent. Use cache, proper locking and merge strategy for best user experience. 😊

8 ORDER SERVICE (LLD)

Order Service is responsible for creating and managing orders from checkout to delivery.

Order Lifecycle (High Level)

Each step has a status & timestamp.

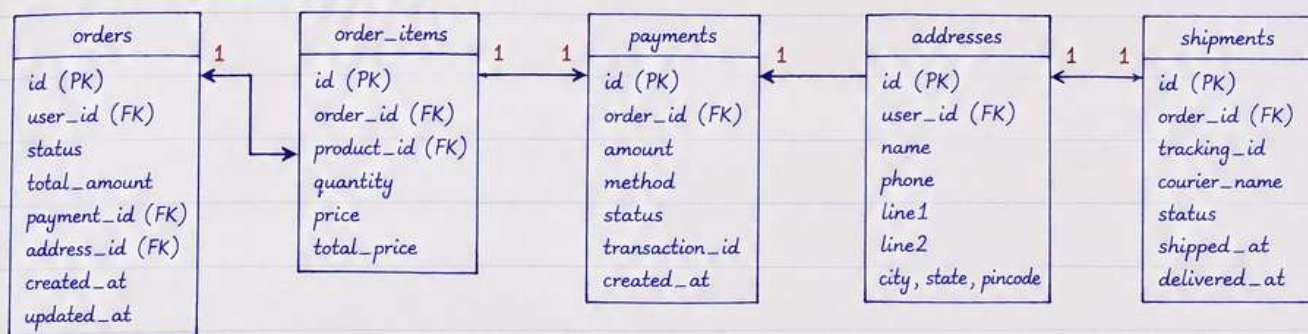
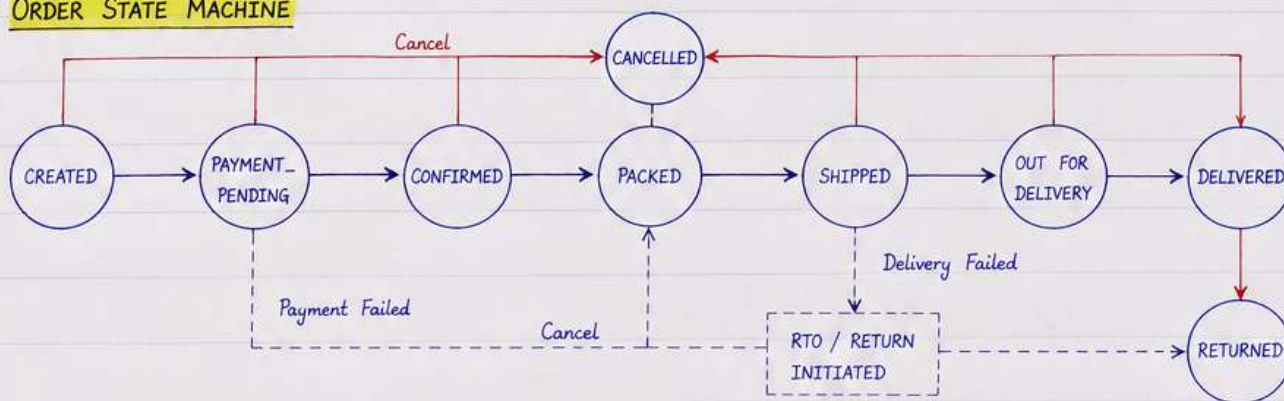
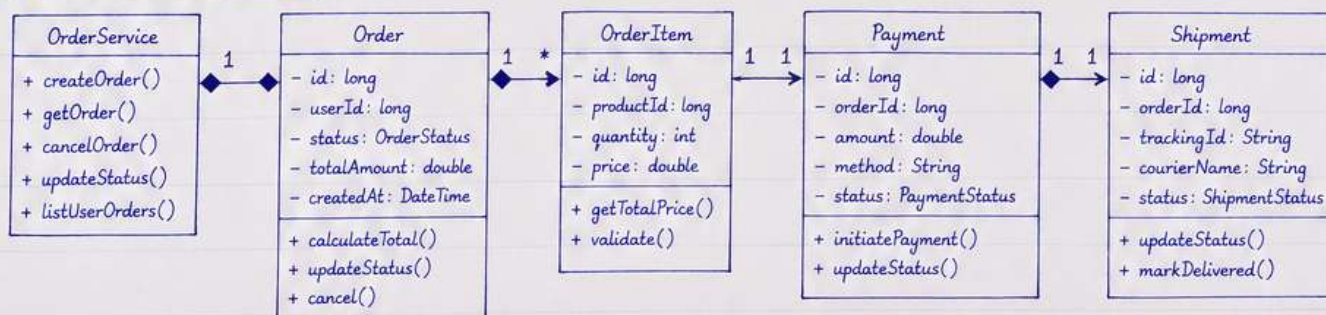
1 ORDER STATES

- **CREATED** - Order is placed by user.
- **PAYMENT_PENDING** - Waiting for payment.
- **CONFIRMED** - Payment successful.
- **PACKED** - Items are packed.
- **SHIPPED** - Order is shipped.
- **OUT_FOR_DELIVERY** - On the way.
- **DELIVERED** - Delivered to customer.
- **CANCELLED** - Order cancelled.
- **RETURNED** - Return initiated/completed.

Order Service must be reliable, consistent and handle retries & failures. 😊

2 ORDER APIs

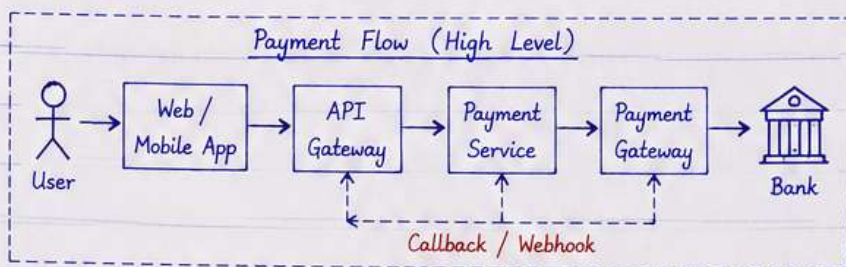
- **POST** /order/create → Create order
- **GET** /order/{orderId} → Get order details
- **PUT** /order/{orderId}/cancel → Cancel order
- **GET** /order/user/{userId} → List user orders
- **PUT** /order/{orderId}/status → Update order status
- **POST** /order/return → Initiate return
- **GET** /order/{orderId}/invoice → Get invoice

3 ORDER DATABASE DESIGN (SQL)**4 ORDER STATE MACHINE****5 CLASS DIAGRAM (LLD)**

* Keep Order Service **idempotent** and handle duplicate requests (use orderId). 😊

9 PAYMENT SERVICE

Payment Service handles all online transactions securely and reliably. It integrates with multiple payment gateways and ensures **idempotency**, **retries** and **status consistency**.

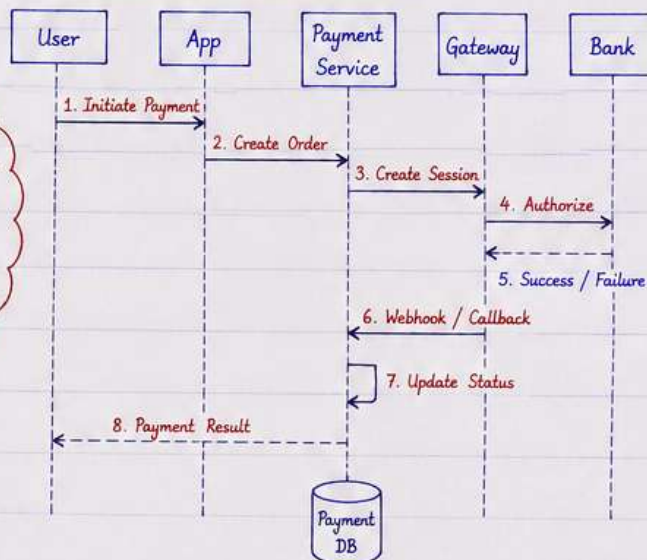


1 PAYMENT GATEWAY INTEGRATION

- Supports multiple gateways (Razorpay, PayU, Stripe, etc.)
- Create order / session with gateway.
- Redirect user to gateway / collect payment details.
- Receive async webhook / callback after payment.
- Update payment status in our system.

2 PAYMENT FLOW (SEQUENCE)

Never trust return URL from gateway. Always verify using server to server callback.



3 IDEMPOTENCY

- Same request multiple times should not create multiple payments.
- Use Idempotency-Key in request header.
- Store key + response in DB/Redis with TTL.
- Return stored response if same key is received again.

Header Example
Idempotency-Key: abc123-uuid

4 RETRY LOGIC

- Network / gateway failures may happen.
- Use exponential backoff retry.
- Retry only for safe operations (create order / verify payment).
- Do not retry after payment success.

Backoff Example
1st Retry → 2 sec
2nd Retry → 4 sec
3rd Retry → 8 sec
Max Retry → 3

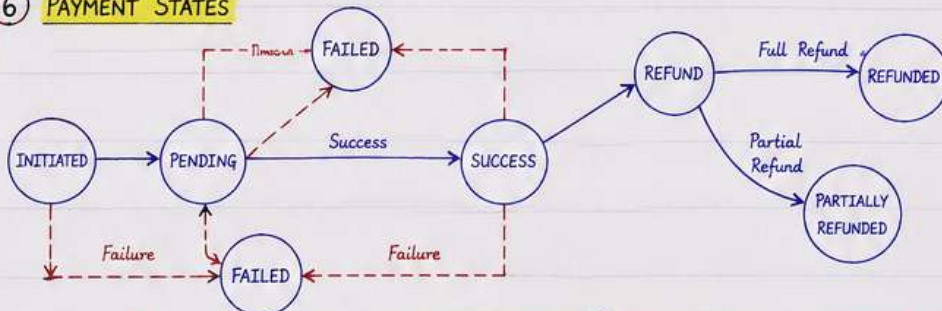
5 WEBHOOK / CALLBACK HANDLING

- Gateway sends async notification.
- Verify signature / secret.
- Update payment status.
- Make it idempotent (same webhook may come multiple times).
- Acknowledge quickly (200 OK).

Verify Signature

- Get header signature
- Generate hash using secret
- Compare hashes
- If invalid → reject

6 PAYMENT STATES



State Meaning

- INITIATED → Payment request created
- PENDING → Waiting for user / gateway
- SUCCESS → Payment successful
- FAILED → Payment failed
- REFUND → Refund process started
- REFUNDED → Full amount refunded
- PARTIALLY REFUNDED → Partial amount refunded

BEST PRACTICES

- ✓ Always validate on server
- ✓ Use HTTPS everywhere
- ✓ Store no card details
- ✓ Log all transactions
- ✓ Monitor payment failures

TABLES (EXAMPLE)

- payments (id, order_id, amount, method, status, gateway_txn_id, created_at, updated_at)
- refund (id, payment_id, amount, status, created_at)

COMMON GATEWAYS

- Razorpay
- PayU
- Stripe
- Cashfree

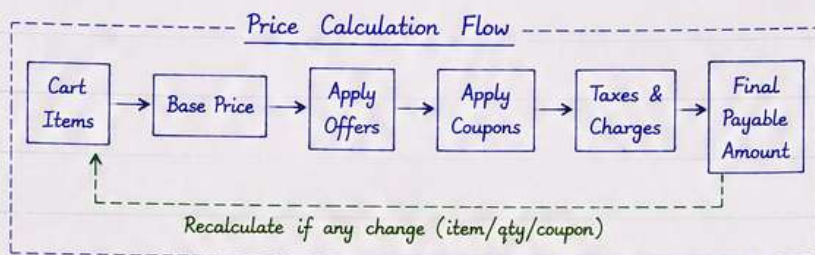


* Payment Service must be **secure**, **reliable** and **idempotent**, 😊

10 PRICE & DISCOUNT SERVICE (LLD)

Price Service calculates final price of product by applying offers, coupons, taxes, delivery charges, etc.

Goal → Fast, accurate & consistent price calculation.

**1 COMPONENTS**

- Offer Service - Provides active offers.
- Coupon Service - Validates & applies coupons.
- Tax Service - Calculates GST based on rules.
- Delivery Service - Calculates delivery charge.
- Price Service - Aggregates all & returns final price.

2 PRICE SERVICE APIS

- POST /price/calculate → Calculate final price
- POST /price/estimate → Estimate price (without stock lock)
- GET /price/breakup → Get price breakup details.
- POST /price/validate-coupon → Validate coupon code
- POST /price/apply-offer → Apply offer to cart

3 PRICE CALCULATION BREAKUP (EXAMPLE)

Base Price (MRP)	₹ 10,000
Offer Discount (10%)	- ₹ 1,000
Coupon Discount	- ₹ 500
Sub Total	₹ 8,500
Delivery Charges	+ ₹ 100
Packaging Charges	+ ₹ 20
GST (18%)	+ ₹ 1,527
Final Payable Amount	₹ 10,147

Always show breakup to user for transparency 😊

NOTE:

Recalculate on any change in cart, offers or address.

11 DATABASE DESIGN (CORE TABLES)

Field	Type
id (PK)	BIGINT
name	VARCHAR
email	VARCHAR
phone	VARCHAR
password_hash	VARCHAR
created_at	DATETIME
updated_at	DATETIME

Field	Type
id (PK)	BIGINT
name	VARCHAR
description	TEXT
category_id (FK)	BIGINT
brand_id (FK)	BIGINT
price	DECIMAL
image_url	VARCHAR
created_at	DATETIME

Field	Type
id (PK)	BIGINT
name	VARCHAR
parent_id (FK)	BIGINT
created_at	DATETIME

Field	Type
id (PK)	BIGINT
user_id (FK)	BIGINT
status	VARCHAR
total_amount	DECIMAL
payment_id (FK)	BIGINT
address_id (FK)	BIGINT
created_at	DATETIME
updated_at	DATETIME

Field	Type
id (PK)	BIGINT
order_id (FK)	BIGINT
product_id (FK)	BIGINT
quantity	INT
price	DECIMAL

Field	Type
id (PK)	BIGINT
order_id (FK)	BIGINT
amount	DECIMAL
method	VARCHAR
status	VARCHAR
transaction_id	VARCHAR
created_at	DATETIME

Field	Type
id (PK)	BIGINT
user_id (FK)	BIGINT
created_at	DATETIME
updated_at	DATETIME

Field	Type
id (PK)	BIGINT
cart_id (FK)	BIGINT
product_id (FK)	BIGINT
quantity	INT
price	DECIMAL

Field	Type
id (PK)	BIGINT
user_id (FK)	BIGINT
name	VARCHAR
phone	VARCHAR
line1	VARCHAR
line2	VARCHAR
city	VARCHAR
state	VARCHAR
pincode	VARCHAR
is_default	BOOLEAN

Field	Type
id (PK)	BIGINT
order_id (FK)	BIGINT
courier_name	VARCHAR
tracking_id	VARCHAR
status	VARCHAR
shipped_at	DATETIME
delivered_at	DATETIME

RELATIONSHIPS (High Level)

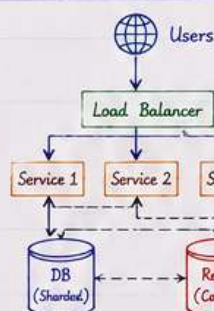
- User 1 → N Orders
- Order 1 → N Order_Items
- Order 1 → 1 Payment
- User 1 → N Addresses
- Order 1 → N Shipments
- Category 1 → N Products



Use Indexing on FKs and search fields.

12 SCALABILITY CONSIDERATIONS

- Use Microservices + Docker + Kubernetes.
- Stateless services behind Load Balancer.
- Cache frequent data in Redis (products, offers, cart, sessions).
- Use CDN for images & static files.
- Database → Read Replicas + Sharding.
- Async processing using Kafka (orders, payments, inventory updates).

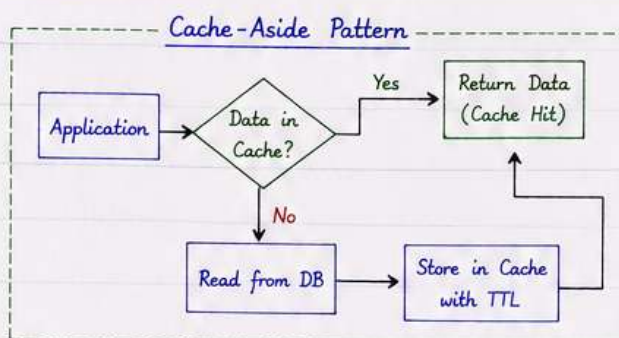
**13 KEY TAKEAWAYS**

- ✓ Break large system into independent services.
- ✓ Design DB carefully (normalize + index).
- ✓ Cache aggressively.
- ✓ Make system scalable, available & reliable.
- ✓ Monitor, log and handle failures gracefully.
- ✓ User experience is the ultimate goal! 😊

* Good design makes system **FAST, SCALABLE & USER FRIENDLY.** 😊

14 CACHING STRATEGY

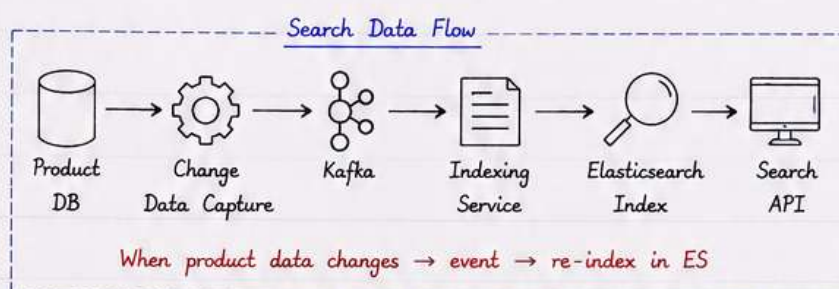
- Use Redis for high read data.
- Cache products, categories, offers, banners, cart, sessions.
- Cache-Aside Pattern.
- Set proper TTL for each key.
- Invalidate / update cache on writes.

Examples of Cache Keys

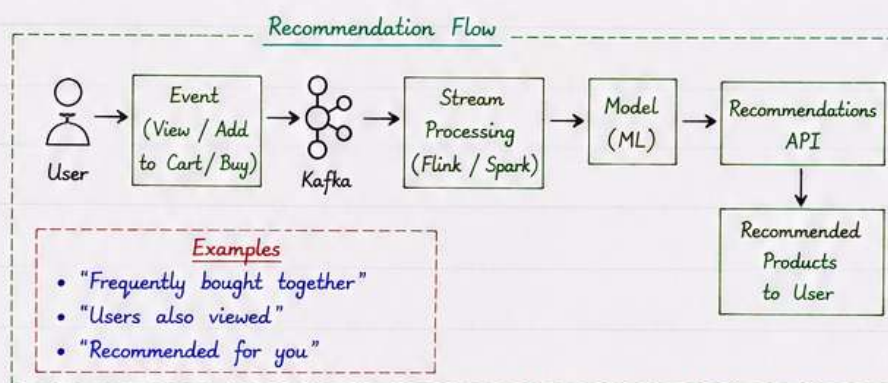
- product: {productId}
- category: {categoryId}
- offer: active
- banner: home
- cart: {userId}
- session: {sessionId}

15 SEARCH SERVICE (ELASTICSEARCH)

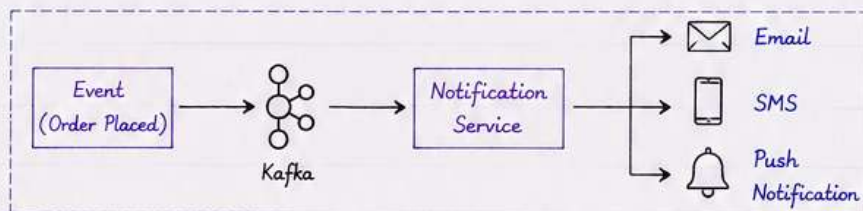
- Products are indexed in Elasticsearch.
- Supports full text search, filters, sorting, facets.
- Auto-complete, typo-tolerance, synonyms.
- Improves search relevance & speed.

**16 RECOMMENDATION SERVICE**

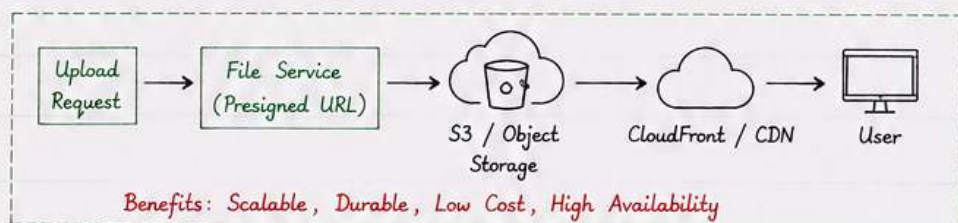
- Personalized recommendations for users.
- Based on browsing history, purchases, ratings, similar users.
- Techniques: Collaborative Filtering, Content Based Filtering, Hybrid.
- Data pipeline: Events → Kafka → Stream Processing → Model → Serve.

**17 NOTIFICATIONS SERVICE**

- Send email, SMS, push notifications.
- Order confirmation, shipping updates, offers, reminders.
- Use templates + queues for scalability.

**18 FILE / MEDIA SERVICE**

- Store product images, videos, documents.
- Upload → S3 / Object Storage (AWS / GCP / MinIO).
- Serve via CDN for fast delivery.

**19 OBSERVABILITY & MONITORING**

- Centralized Logging: ELK Stack (Elasticsearch + Logstash + Kibana)
- Metrics: Prometheus + Grafana
- Tracing: Jaeger / Zipkin
- Alerts: PagerDuty / Opsgenie
- Track: Latency, Error rate, Throughput, DB slow queries, Queue lag, etc.

**20 SECURITY CONSIDERATIONS**

- Authentication: OAuth2 / JWT
- Authorization: RBAC
- Data Encryption: In-transit (HTTPS), At-rest (AES)
- Secure APIs: Rate limiting, Captcha, Input validation
- PCI-DSS for payments
- Protect against XSS, CSRF, SQL Injection
- WAF & DDoS Protection



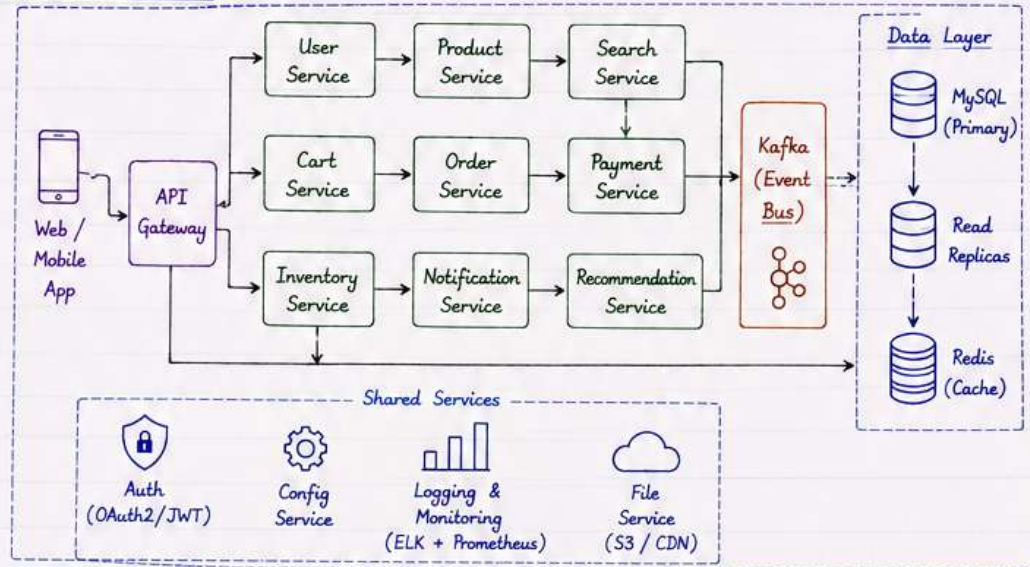
* **TAKEAWAY:** Design for scale, reliability and performance.

Always keep **User Experience** & **Business Goals** at the center. 😊



21 HIGH LEVEL ARCHITECTURE OVERVIEW

- Microservices based architecture.
- Each service is independent, scalable and fault tolerant.
- API Gateway is single entry point.
- Data is cached, replicated and sharded.
- Async communication via Kafka for high throughput.



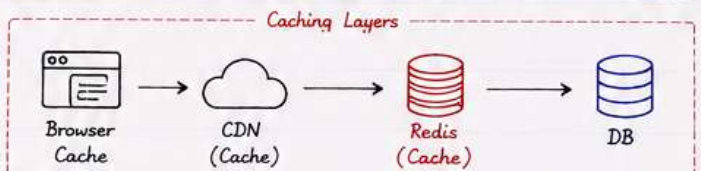
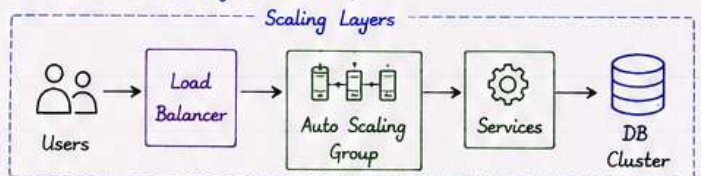
22 DATABASE STRATEGY

- Use SQL for transactional data.
- Use Read Replicas for read scaling.
- Use Sharding for large tables (orders, users).
- Use Redis for caching and sessions.
- Use Elasticsearch for search.

Use Case	Primary DB	Cache/Search	Notes
Users, Orders, Cart, Payments	MySQL	Redis	ACID Transactions
Products, Categories	MySQL	Redis + ES	Read Heavy
Search	Elasticsearch	-	Full text search
Sessions, Tokens	-	Redis	Fast access

23 SCALABILITY APPROACH

- Horizontal scaling of microservices.
- Auto scaling based on traffic.
- Load Balancer distributes requests.
- Caching at multiple layers.
- Database sharding & read replicas.



24 RELIABILITY & FAULT TOLERANCE

- Redundant instances across AZ / regions.
- Health checks & auto recovery.
- Retry with exponential backoff.
- Circuit Breaker pattern for dependencies.
- Data backup and point-in-time recovery.

Fault Tolerance Patterns

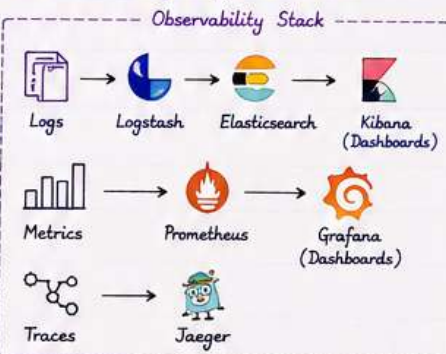
- ★ Retry
- ★ Circuit Breaker
- ★ Timeouts
- ★ Bulkhead Isolation
- ★ Idempotency

Backup Strategy

- ★ Automated Daily Backups
- ★ Cross Region Replication
- ★ Point-in-Time Recovery
- ★ Disaster Recovery Plan

25 MONITORING & OBSERVABILITY

- Monitor logs, metrics & traces.
- Set alerts for failures & latency.
- Dashboards for system health.



Key Metrics

- ★ Request Rate (RPS)
- ★ Error Rate (4xx/5xx)
- ★ Latency (P50/P95/P99)
- ★ DB CPU, Connections
- ★ Cache Hit Ratio
- ★ Order Success Rate

26 INTERVIEW TIPS

Points to Highlight in Interview

- ✓ Clearly define scope & assumptions.
- ✓ Break down the system into components.
- ✓ Discuss APIs, DB design and data flow.
- ✓ Explain scalability, reliability & monitoring.
- ✓ Mention trade-offs you considered.

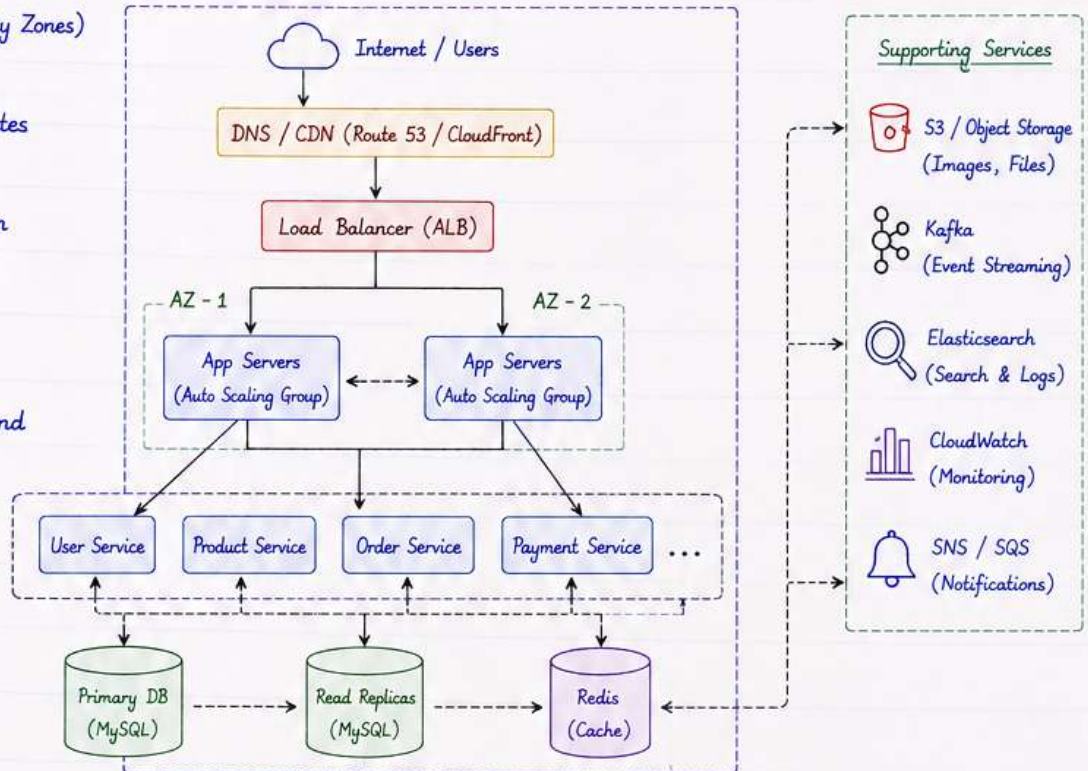
Trade-offs

- Consistency vs Availability (CAP Theorem)
- SQL vs NoSQL (Relational vs Flexible Schema)
- Cache TTL - Freshness vs Performance
- Microservices - Complexity vs Scalability

Always design for **SCALE, RELIABILITY** and **GREAT USER EXPERIENCE!** 😊

27 DEPLOYMENT ARCHITECTURE (HIGH LEVEL)

- Multiple AZ (Availability Zones) for high availability.
- Load Balancer distributes traffic.
- Auto Scaling based on demand.
- Stateless services for scalability.
- Monitoring, Logging and Alerts everywhere.



28 BEST PRACTICES

- ✓ Design for scale from day one.
- ✓ Keep services stateless.
- ✓ Cache aggressively.
- ✓ Asynchronous communication wherever possible.
- ✓ Monitor, log and set alerts.
- ✓ Handle failures gracefully.
- ✓ Secure all data and APIs.
- ✓ Optimize for cost and performance.

Focus on
User Experience,
Performance,
Cost Optimization
and
Reliability.



30 COMMON CHALLENGES

- Handling flash sales and traffic spikes.
- Inventory consistency and overselling.
- Payment failures and retries.
- Shipping delays and tracking.
- Fraud detection and security.
- Multi-region data consistency.



31 KEY TAKEAWAYS

- ★ Break the system into independent, scalable microservices.
- ★ Use caching, CDN and search to improve performance.
- ★ Design database and infrastructure for high availability.
- ★ Asynchronous processing for heavy and non-blocking tasks.
- ★ Monitor, measure and improve continuously.

29 TECHNOLOGY STACK (EXAMPLE)

Frontend	: React / Next.js
Backend	: Java / Node.js / Go
API Gateway	: Kong / AWS API Gateway
Databases	: MySQL (Primary + Read Replicas)
Cache	: Redis / Memcached
Search	: Elasticsearch
Messaging	: Kafka / SQS / SNS
Storage	: S3 / GCS
Monitoring	: Prometheus + Grafana / CloudWatch
Logging	: ELK Stack (Elasticsearch + Logstash + Kibana)
Deployment	: Docker + Kubernetes (EKS / GKE)
CDN	: CloudFront / Cloud CDN
Cloud Provider	: AWS / GCP



Great Design → Great Experience → Happy Users 😊

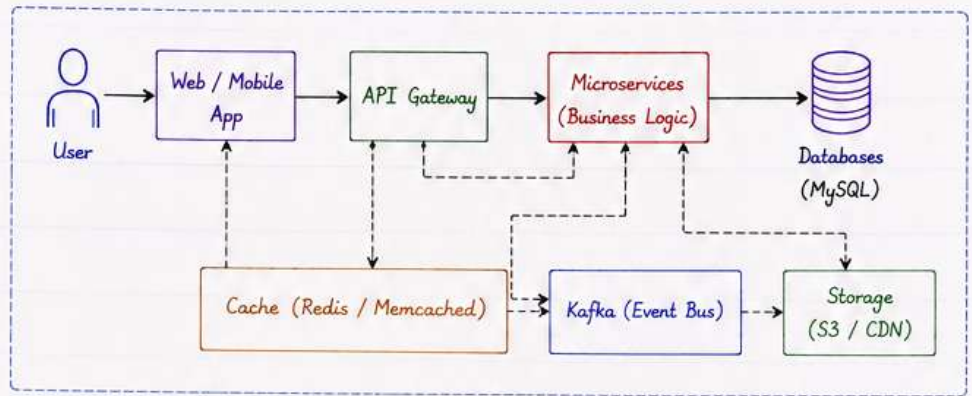


KEEP LEARNING, KEEP BUILDING and KEEP SHARING! You've Got This! 😊



32 DATA FLOW SUMMARY

Here's the high level data flow of the entire E-Commerce platform.



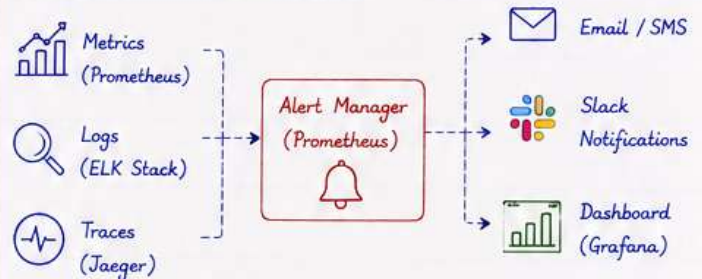
33 DESIGN TRADE-OFFS

Trade-Off	Pro	Con
SQL vs NoSQL	SQL → ACID, complex queries	NoSQL → Scalability, flexibility
Cache (Redis)	Low latency, high performance	Data may be stale, extra infra
Microservices	Independent deploy, scalable	Complexity, network communication
Event Driven (Kafka)	Decoupled, high throughput	Eventual consistency, more components
CDN (CloudFront)	Faster content delivery, reduces load	Extra cost, cache invalidation

36 KEY DESIGN PATTERNS USED

- API Gateway Pattern → Single entry for all clients.
- Microservices Pattern → Services are small & independent.
- Event Driven Pattern → Use Kafka for async workflows.
- Cache-Aside Pattern → Check cache first, then DB.
- Database Sharding → Horizontal scaling of DB.
- CQRS (Where needed) → Separate read & write models.
- Bulkhead Pattern → Isolate failures to a component.
- Idempotent Processing → Handle retries safely.

37 MONITORING & ALERTING



34 FAILURE HANDLING

- Retry with exponential backoff.
- Circuit Breaker to stop cascading failures.
- Fallback / Default responses.
- Health checks and auto recovery.
- Data replication & backups.



35 ESTIMATING CAPACITY (EXAMPLE)

- DAU (Daily Active Users) : 10 Million
- Read / User / Day : 50
- Write / User / Day : 10
- Total Reads / Day : 500 Million
- Total Writes / Day : 100 Million
- Read / Sec : ~ 5800
- Write / Sec : ~ 1150

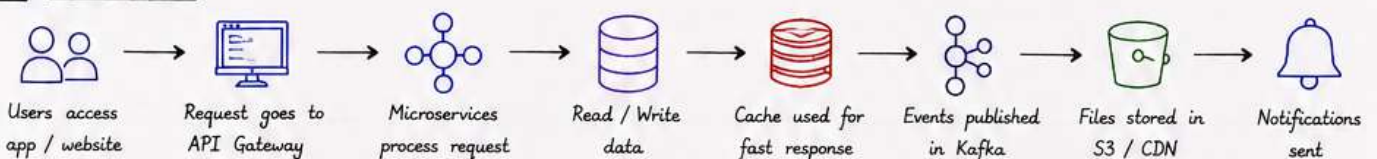
Always estimate for peak traffic (2x - 3x)

38 FUTURE IMPROVEMENTS

- ★ AI based recommendations and personalization.
- ★ Real time inventory updates using streams.
- ★ Global expansion with multi-region deployment.
- ★ More automation in scaling and deployments.
- ★ Advanced analytics and business intelligence.
- ★ Improve search with ML based ranking.

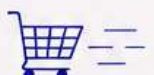


39 QUICK RECAP



Break down → Design components → Handle scale → Ensure reliability → Monitor & Improve 😊

★ Great Systems are built step by step. Keep Learning, Keep Building! 📚😊



40 SECURITY BEST PRACTICES

- Use HTTPS everywhere.
- Store passwords using Hashing (bcrypt).
- JWT for Authentication.
- RBAC for Authorization.
- Validate all inputs.
- Sanitize data to prevent injection attacks.
- Rate limiting on APIs.
- WAF & DDoS protection.
- Regular security audits & dependency updates.



41 COST OPTIMIZATION TIPS

- Right size your instances.
- Use auto scaling.
- Use spot/preemptible instances.
- Enable caching (Redis / CDN).
- Archive old data to S3 Glacier.
- Monitor & remove unused resources.
- Use serverless for spiky workloads.
- Optimize queries & indexes.

Cost Saving Formula

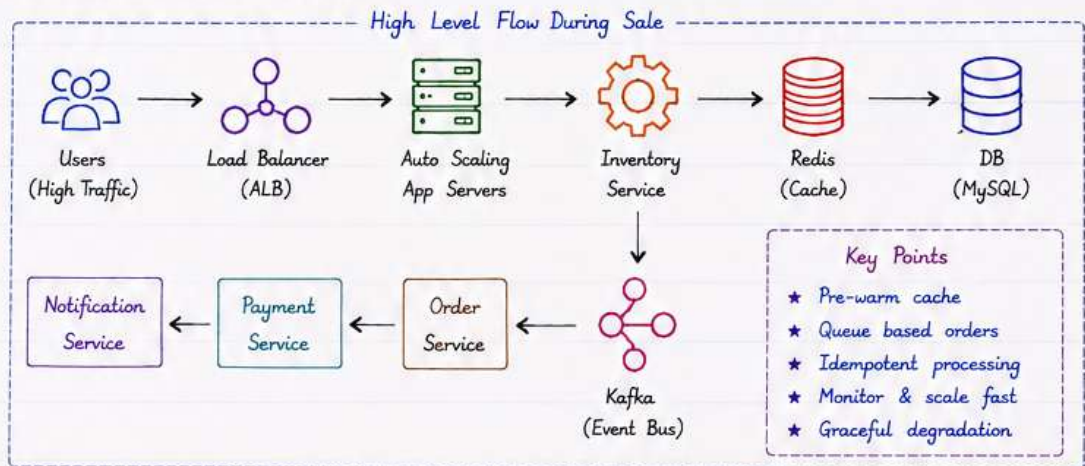
$$\text{Lower Infra Cost} + \text{Efficient Architecture} + \text{Right Tool for the Job} =$$

Optimal Cost
High Performance



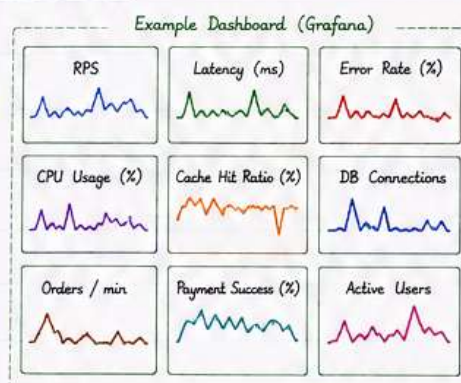
42 CASE STUDY : FLASH SALE / BIG BILLION DAY

- Sudden traffic spike (10x - 100x).
- Inventory & payment consistency.
- Avoid over selling.
- Fast response & high availability.
- Queue based async processing.



43 COMMON METRICS TO TRACK

- Request Rate (RPS)
- Error Rate (4xx / 5xx)
- Latency (P50 / P95 / P99)
- DB CPU & Connections
- Cache Hit Ratio
- Order Success Rate
- Payment Success Rate
- Inventory Consistency
- Active Users (DAU / MAU)

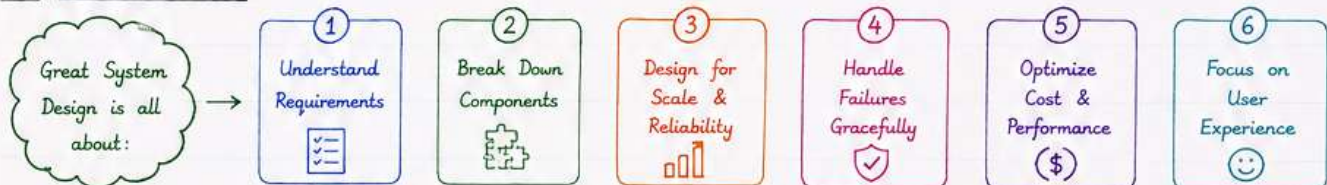


44 TOOLS & TECHNOLOGIES (EXAMPLE)

- Language : Java / Go / Node.js
- Framework : Spring Boot / NestJS
- Databases : MySQL, MongoDB
- Cache : Redis / Memcached
- Messaging : Kafka / RabbitMQ / SQS
- Search : Elasticsearch
- Storage : S3 / GCS
- CDN : CloudFront / Cloud CDN
- Monitoring : Prometheus + Grafana
- Logging : ELK Stack
- Container : Docker
- Orchestration : Kubernetes
- CI/CD : Jenkins / GitHub Actions



45 FINAL THOUGHTS



* KEEP PRACTICING, KEEP BUILDING, KEEP SHARING !!

Remember: Simplicity First, Then Scale. Happy Coding!

